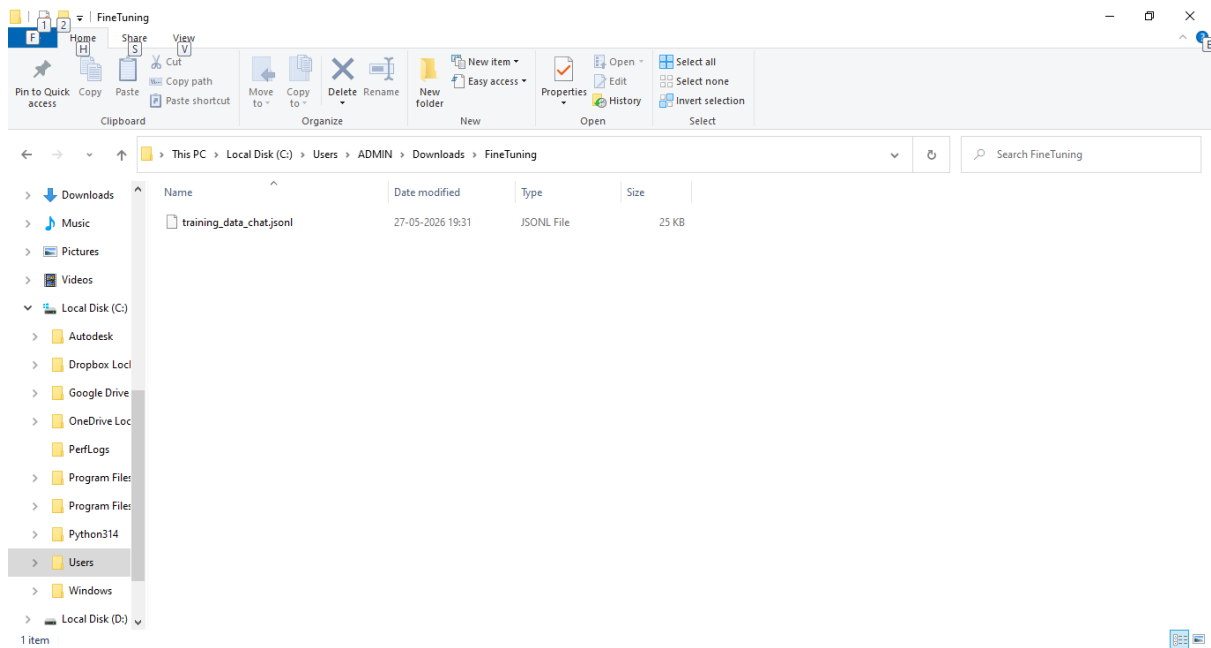
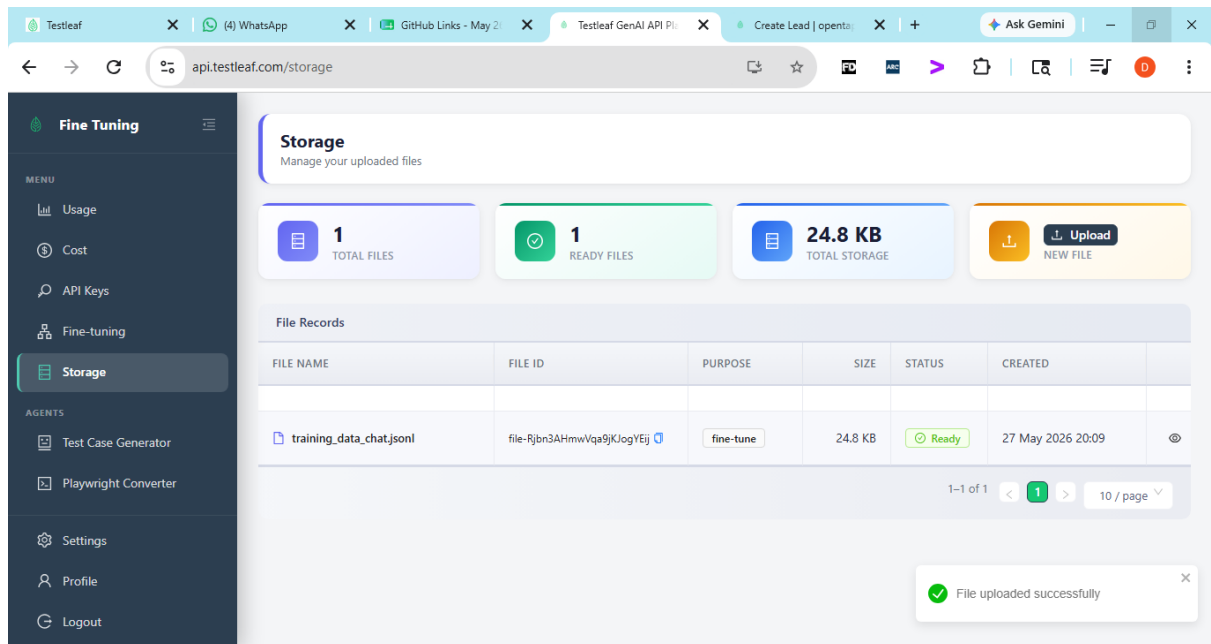


Data in .Jsonl format



Uploading the dataset in storage



Create the Fine tune model

The screenshot shows the Testleaf Fine-tuning interface with a modal form titled "Create a fine-tuned model". The form contains the following fields and options:

- METHOD:** Supervised
- BASE MODEL:** gpt-4o-mini-2024-07-18 (Note: Base model is required.)
- TRAINING DATA:** Select existing (file-Rjbn3A4hmWQa9KlogYEJ)
- VALIDATION DATA:** None
- SEED:** 0 (Controls reproducibility. Same seed + parameters produces the same result. Leave 0 to auto-generate.)
- HYPERPARAMETERS:**
 - Batch Size: 1
 - Learning Rate Multiplier: 0.20 (Recommended range: 0.0001 - 10)

Buttons: Cancel, Create Model

Fine Tune Model created successfully

The screenshot shows the Testleaf Fine-tuning interface after a successful model creation. The summary statistics are:

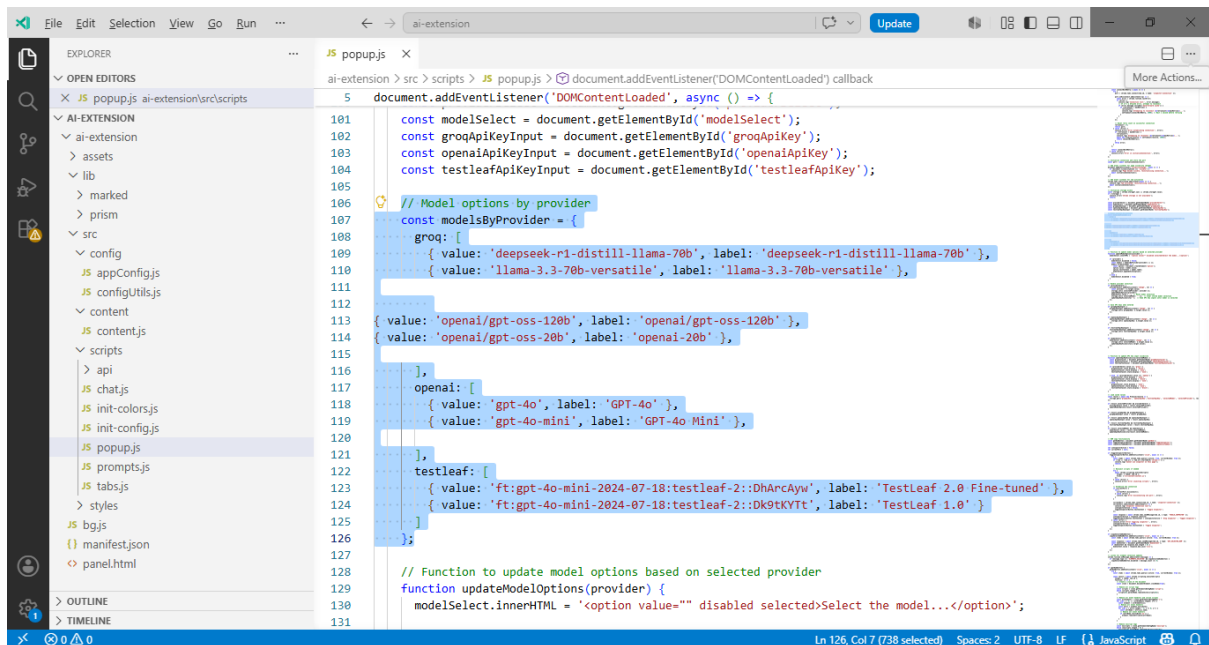
- TOTAL JOBS: 1
- SUCCEEDED: 1
- FAILED: 0
- TRAINED TOKENS: 21,316

The table below lists the fine-tuned models:

FINE-TUNE ID	BASE MODEL	METHOD	STATUS	TRAINED TOKENS	CREATED
ftgpt-4o-mini-2024-07-18-testleaf-2-Dk9KYt	gpt-4o-mini-2024-07-18	supervised	Succeeded	21,316	27 May 2026 20:15

Page 1 of 1, 10 / page

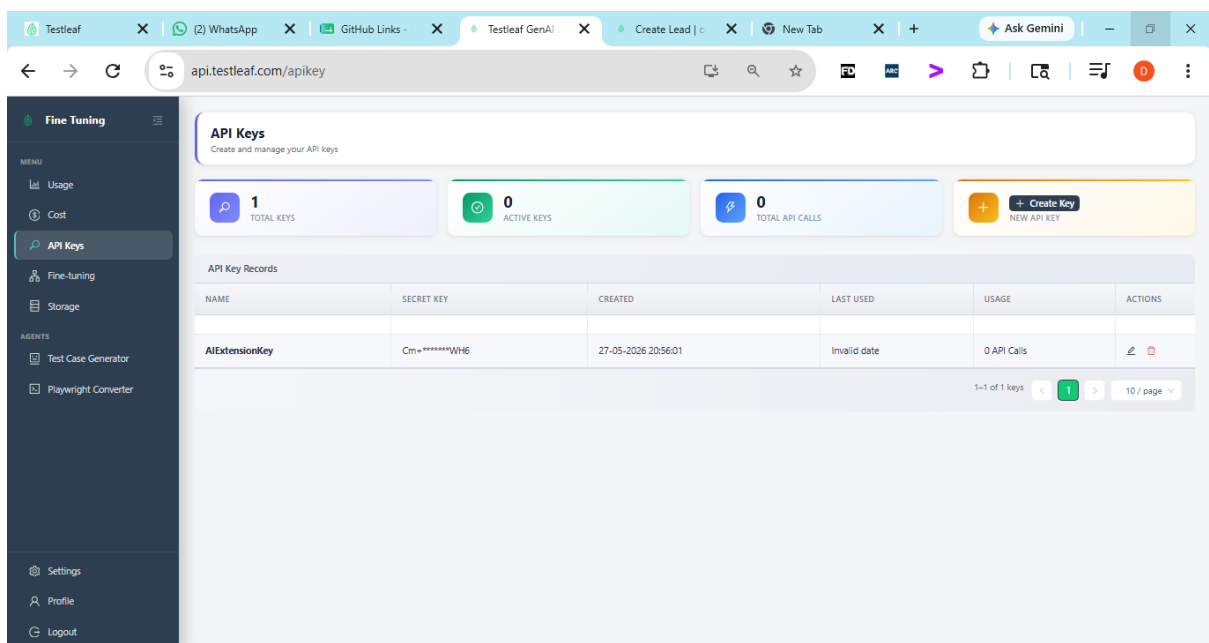
In popup.js implement modelsByProvider under AI extension



The screenshot shows a VS Code editor with the file explorer on the left and the editor window in the center. The file explorer shows the project structure with folders like 'ai-extension' and 'src'. The editor window shows the file 'popup.js' with the following code:

```
5 document.addEventListener('DOMContentLoaded', async () => {
101 const modelSelect = document.getElementById('modelSelect');
102 const groqApiKeyInput = document.getElementById('groqApiKey');
103 const openaiApiKeyInput = document.getElementById('openaiApiKey');
104 const testleafApiKeyInput = document.getElementById('testleafApiKey');
105
106 // Model options by provider
107 const modelsByProvider = {
108   groq: [
109     { value: 'deepseek-r1-distill-llama-70b', label: 'deepseek-r1-distill-llama-70b' },
110     { value: 'llama-3.3-70b-versatile', label: 'llama-3.3-70b-versatile' },
111   ],
112   openai: [
113     { value: 'openai/gpt-oss-120b', label: 'openai/gpt-oss-120b' },
114     { value: 'openai/gpt-oss-20b', label: 'openai-20b' },
115   ],
116   testleaf: [
117     { value: 'gpt-4o', label: 'GPT-4o' },
118     { value: 'gpt-4o-mini', label: 'GPT-4o Mini' },
119   ],
120   testleaf: [
121     { value: 'ft:gpt-4o-mini-2024-07-18:testleaf-2::DhArcAyyw', label: 'TestLeaf 2.0 Fine-tuned' },
122     { value: 'ft:gpt-4o-mini-2024-07-18:testleaf-2::Dk9tKYtt', label: 'TestLeaf 1.0' }
123   ],
124 ];
125
126 // Function to update model options based on selected provider
127 function updateModelOptions(provider) {
128   modelSelect.innerHTML = '<option value="" disabled selected>Select the model...</option>';
129 }
```

API key is created and added in the extension



The screenshot shows the Testleaf API Keys management interface. The page title is 'API Keys' and the subtitle is 'Create and manage your API keys'. The interface includes a sidebar with a menu containing 'Usage', 'Cost', 'API Keys', 'Fine-tuning', 'Storage', 'AGENTS', 'Test Case Generator', and 'Playwright Converter'. The main content area shows the 'API Keys' section with a summary of 1 total key, 0 active keys, and 0 total API calls. A 'Create Key' button is visible. Below this, there is a table of API Key Records with columns: NAME, SECRET KEY, CREATED, LAST USED, USAGE, and ACTIONS. The table contains one record for 'AIExtensionKey' with a secret key 'Cm*****WH6', created on '27-05-2026 20:56:01', and last used on 'Invalid date'. The usage is '0 API Calls'. The table has a pagination bar at the bottom showing '1-1 of 1 keys' and '10 / page'.

NAME	SECRET KEY	CREATED	LAST USED	USAGE	ACTIONS
AIExtensionKey	Cm*****WH6	27-05-2026 20:56:01	Invalid date	0 API Calls	Edit Delete

Testleaf X (2) WhatsApp X GitHub Links X Testleaf GenAI X Create Lead | c X Login | Salesfo X + Ask Gemini

login.salesforce.com



Salesforce login

Username

Password

Log In

☐ Remember me

[Forgot Your Password?](#) [Use Custom Domain](#)

or

☐ Log In with Email

AI Browser Extension for Code Generation

Code Generator

Supported Language

Java

Browser Engine

Selenium

Select LLM Provider

Testleaf

Select Model

TestLeaf 1.0

Testleaf API Key

Settings

Testleaf X (2) WhatsApp X GitHub Links X Testleaf GenAI X Create Lead | c X Login | Salesfo X + Ask Gemini

login.salesforce.com



Salesforce login

Username

Password

Log In

☐ Remember me

[Forgot Your Password?](#) [Use Custom Domain](#)

or

☐ Log In with Email

AI Browser Extension for Code Generation

Code Generator

☒ Feature File ☒ Page Class

[Reset Chat](#)

Feature: Login to Salesforce

Scenario Outline: Successful login with valid credentials

Given I open the login page

When I type "<username>" into the Username field

And I type "<password>" into the Password field

And I click the Login button

Then I should be logged in successfully

Examples:

username	password
"user1@example.com"	"Password123"
"user2@example.com"	"Password456"
"user3@example.com"	"Password789"

Settings